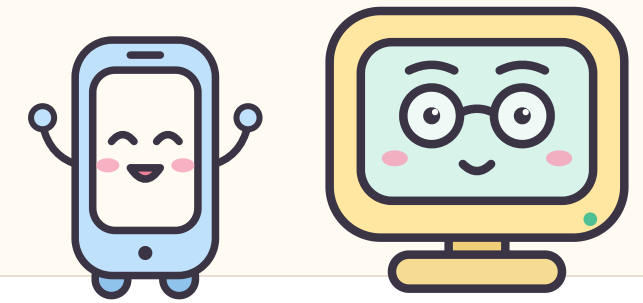




第 22 集 · 约 21 分钟

终章：从第一性原理看架构

从几条推不翻的事实出发，推出主要的设计

配套阅读：书的第 11 章



从零写出 LanShare

这一集你会学到

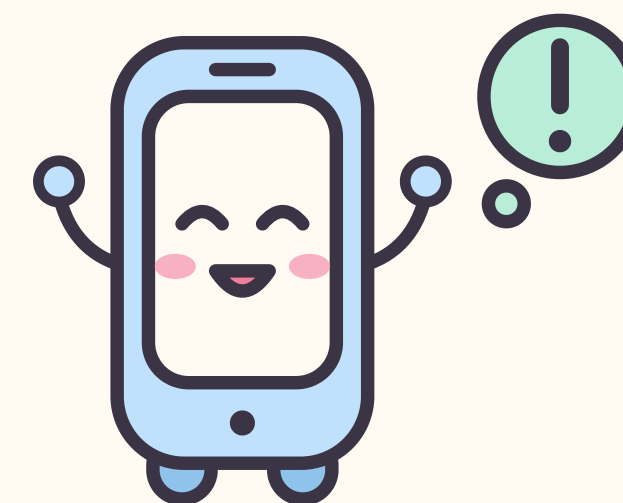
1. 什么是第一性原理：不照搬“别人都这么做”，从推不翻的事实往上推
2. 六条事实，推出 LanShare 的主要设计：边界、上限、锁、纯核心、测试、诚实
3. 推论之间会打架：谁让谁，也要讲清楚
4. 每个决定都有代价；以后设计你自己的系统，照七个步骤走



一个问题

架构，是一份“最佳实践”清单吗？

不是。架构是从几条推不翻的事实，一步一步推出来的决定。





什么是第一性原理

不是“别人这么做，所以我也这么做”，而是“事实是这样，所以只能这么做”

🔴 类比：大家都这么做

例如“网页要安全，就上 HTTPS”

🟢 第一性原理：从推不翻的事实往上推

在你的场景里，怎么选都改变不了的限制：物理、经济、人，也包括平台的规则

🟡 LanShare 里的一个例子

事实：局域网地址不是安全上下文，浏览器不给加密接口；目标：手机只用浏览器 → 得自己带加密代码



事实一：网络是公共的

同一个 WiFi 里的人，看得见你发的东西，也发得出假的请求

● 推论：内容在离开设备之前就加密

端到端加密；密钥放在二维码的

● 推论：画出信任边界，每一道边界都有人看门

网络（Host、Origin）→ 密钥（握手）→ 设备（新设备确认）；另有一类操作只认本机

● 推论：看门的地方要少、要集中

加密请求只从 sealed 一个入口进来；“只认本机”的检查还分散在 5 个函数里，靠测试守着

● 放到 Python 项目里

中间件最集中；装饰器省了重复代码，但每个视图还得记得挂上



事实二：资源有限，而攻击几乎不要钱

发一个请求不花钱，处理一个请求要花内存、时间和磁盘

● 推论：便宜的检查放前面，贵的放后面

让每个假请求，在你这里只花最少的成本

● 推论：会增长的，要有上限；会等待的，要有时限

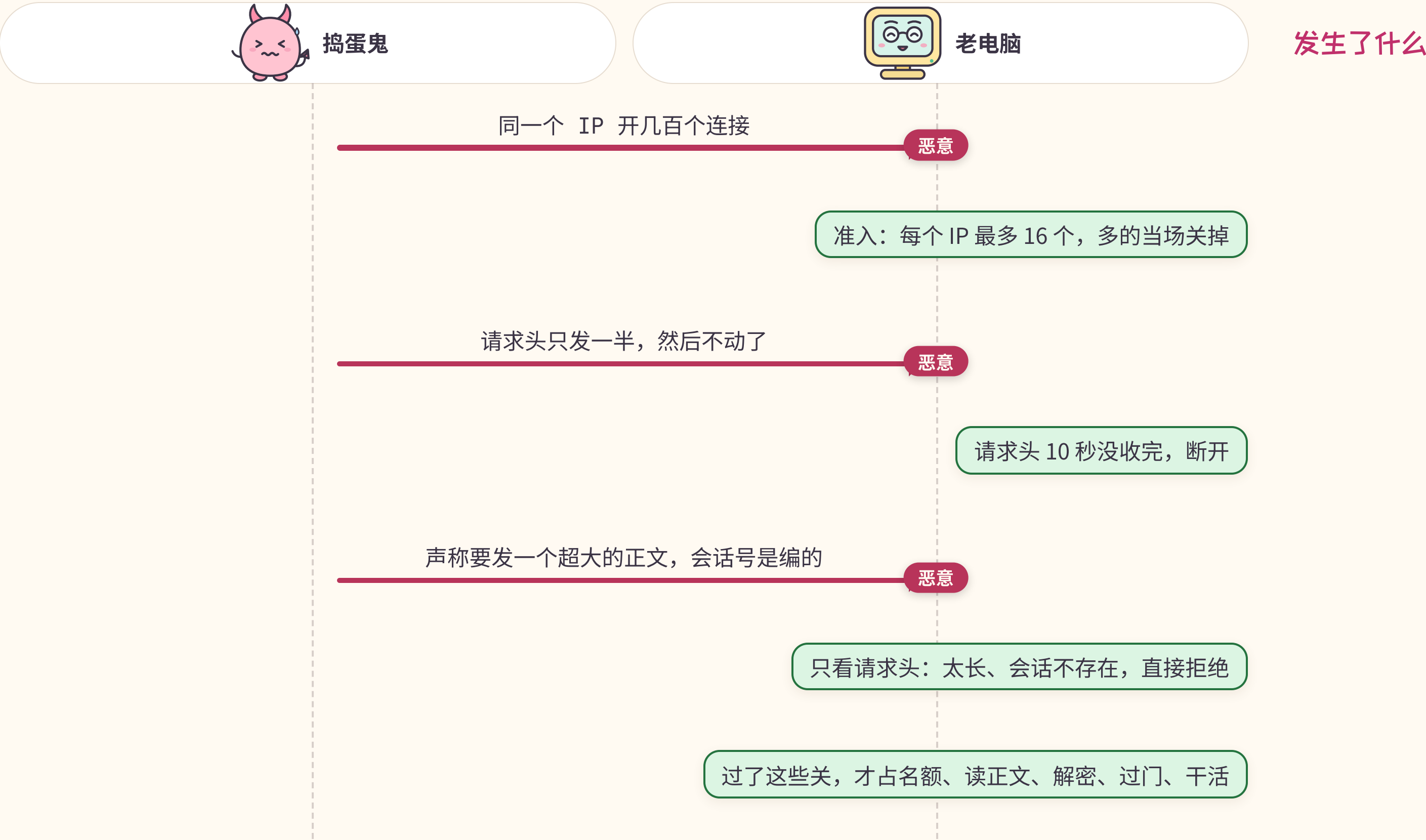
没有上限的东西，迟早会被人用满

● 放到 Python 项目里

先看请求头里的长度再读正文；requests 要写 timeout；缓存别用没有上限的 `functools.cache`



便宜的关在前：捣蛋鬼耗资源





主要的上限和时限

东西	上限	时限
连接	全部 256 个；每个 IP 16 个；本机另有 32 个	空闲 60 秒断开
握手请求	正文最多 4 KiB	10 秒内收完
加密请求的正文	同时 32 个；每个 IP 8 个	5 分钟内读完
会话	256 个，满了淘汰最久没用的	没有空闲时限；程序重启全部作废
新开会话	每个 IP 每分钟 30 个；全局 300 个	按 60 秒计数
上传	同时 64 个；每个会话 4 个	10 分钟没有新块就作废
口令尝试	每个 IP 每分钟 5 次；全局 20 次；累计 30 次就暂停	按 60 秒计数；暂停不会自动解除
待确认的设备	8 台	60 秒没再问就撤掉





事实三：很多事会同时发生



两个请求可能在同一个瞬间到达，而它们看到的是同一份状态

● 推论：先查后改，中间不能有缝

最后那一次“检查”和“登记”，在同一次加锁里完成

● 推论：锁里只算，慢事出了锁再做

例外：查和改本身就是慢事时，用一把只管这件事的锁

● 放到 Python 项目里

with lock 只管得住同一个进程；数据库靠唯一约束、ON CONFLICT，或者事务里加行锁



一个例子：重放的票，只能检一次

crates/lanshare/src/server/channel.rs

```
1  /// 解密正文并登记计数器。
2  fn open_body(st: &super::AppState, cid: [u8; 16], ctr: u64, keys: &ChannelKeys, path: &str, body: &[u8]) -> Result<Opened, StatusCode> {
3      let plain = proto::open(&keys.c2s, ctr, &proto::req_aad(path), body).map_err(|_| StatusCode::FORBIDDEN)?;
4      // 第二次加锁：再查一次再登记，两个相同的重放请求并发到达时只有一个能通过。
5      let mut sessions = st.sessions.lock().expect("锁不会中毒");
6      let Some(session) = sessions.get(&cid) else {
7          return Err(StatusCode::UNAUTHORIZED);
8      };
9      if !session.window.check(ctr) {
10         return Err(StatusCode::FORBIDDEN);
11     }
12     session.window.commit(ctr);
```



事实四：外部世界不确定，规则是确定的

时间、随机数、网络、操作系统、界面，又难测又常变；协议的规则，给定输入就有确定的输出

● 推论：核心要纯，外壳要薄

协议核心不碰网络、文件、时钟，连随机数都由调用方传进来

● 推论：同一件事只有一个真相

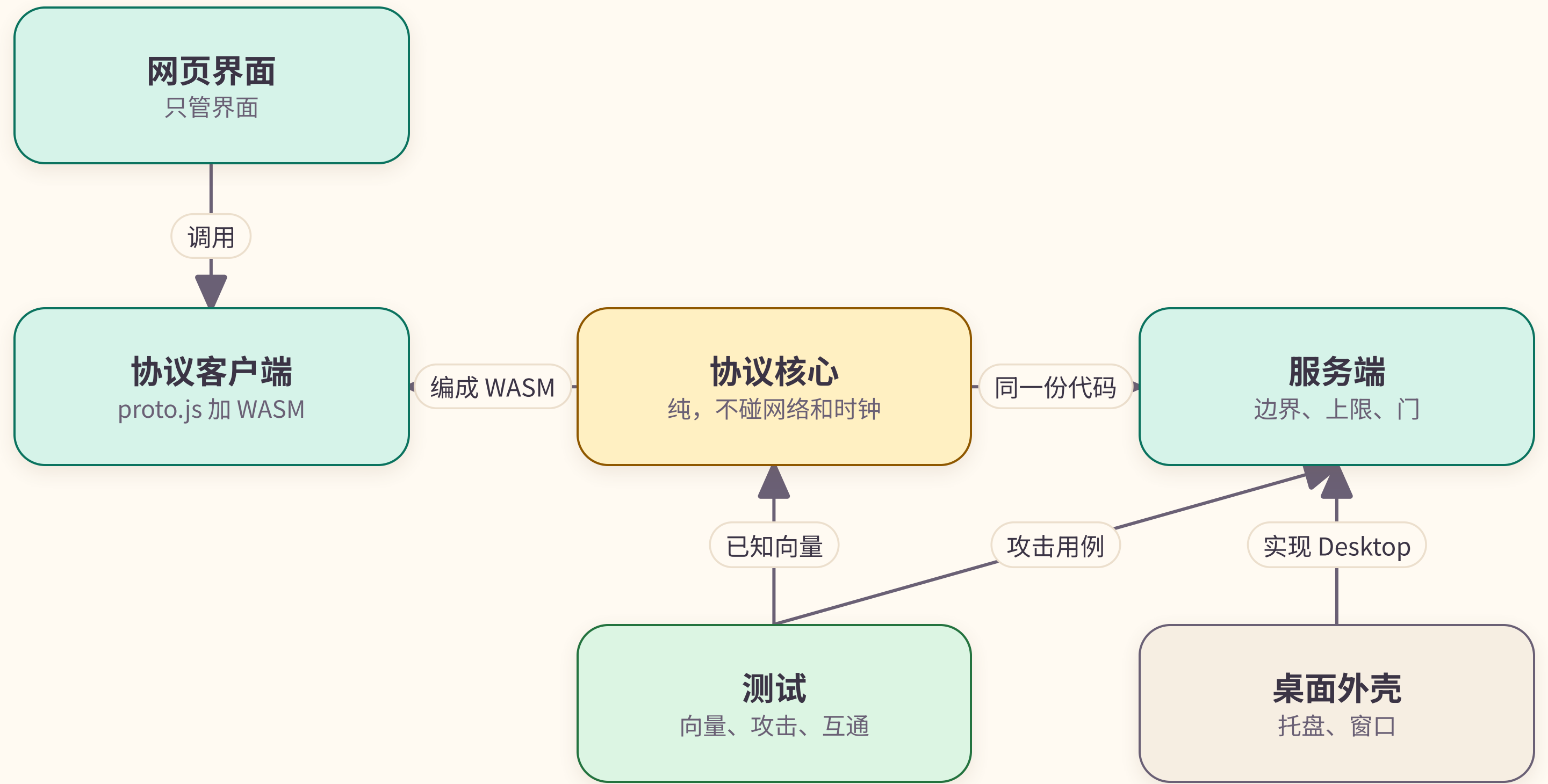
加密只写一遍；已知向量由另一套实现生成；写了两遍的地方，用互通测试对齐

● 放到 Python 项目里

计算写成纯函数，时间、随机数、读到的内容当参数传进去；规则和常量只放一处



推出来的结构





事实五：人会犯错，包括你自己

你自己写的测试，只覆盖你自己想到的情况

● 推论：用独立的东西来核对

另一套实现生成的向量；没有参与开发的独立审查者

● 推论：证明测试真的会响

变异测试：把修复故意改回去，对应的测试必须变红

● 推论：改动要小，旧测试原样通过

新功能有开关，正式运行默认落在安全的一边

● 放到 Python 项目里

用标准文档里的测试向量；请没写这段代码的人来挑刺；新功能用开关接上，默认安全



事实六：最后做判断的是人



用户要知道能信什么、不能信什么；有些决定，只有人能做

● 推论：照实写出边界

安全说明里专门一节“不防什么”；只说“防窃听”，不说“绝对安全”

● 推论：把关键决定交给别人，并且给人核对的依据

新设备要电脑前的人点头，还有 4 位验证码可核对；口令被暂停后，只能在电脑上重新开启

● 放到 Python 项目里

README 里写清楚限制；危险操作要人确认，并且给出确认的依据



每个决定都有代价

决定	换来的	付出的
不用 wasm-bindgen	依赖少，没有工具版本要对齐	自己管线性内存和裸指针
不用 axum 自带的 serve	请求头有时限，每个 IP 单独限额	自己写接收连接的循环
只用浏览器、不装 App	手机扫码就能用	页面每次从网络加载，挡不住主动中间人
密钥只活一次运行	重启就能让所有设备失效	每次重启都要重新扫码
新设备按 IP 放行	协议不用改，刷新页面不用再点	共用一个 IP 的设备会一起放行



以后设计你自己的系统：七个步骤

每一步都写下一样东西；不背清单，每次都从事实推一遍

一、写下目标和非目标

为谁、解决什么问题；明确不解决什么

二、写下事实清单

网络、资源、并发、外部世界、人会犯错、由人判断，再加上你场景特有的限制

三、画出边界图

谁可信、数据从哪来、每道边界在哪段代码里检查

四、写威胁和故障清单

没有密钥的人能做什么？拿到密钥的人能做什么？什么会无限增长、无限等待？最便宜的攻击是什么？

五、写决定和代价表

每个决定追到哪条事实、换来什么、付出什么；追不到事实的，多半是照搬

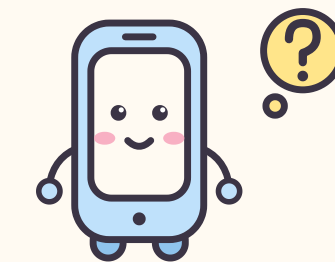
六、写测试清单

每个决定由哪个测试守着；把修复故意改回去，测试必须变红；数字要么实测，要么写清怎么算

七、写下不防什么，并给演进留余地

照实写边界；核心纯、外壳薄；改动小、旧测试原样通过；新功能有开关，默认安全





选择题

下面哪一种，才是从第一性原理出发的理由？

- A 大公司都用这个框架，所以我们也用
- B 手机用局域网地址打开页面时，浏览器不给加密接口，而我们又要求手机只用浏览器，所以得自己带一份加密代码
- C 这个写法看起来更优雅



选择题

下面哪一种，才是从第一性原理出发的理由？

A 大公司都用这个框架，所以我们也用

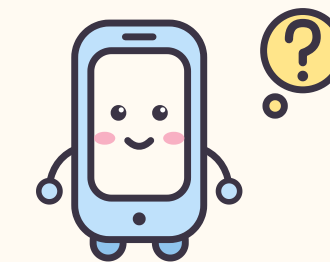
这是类比。大公司的场景和限制，可能和你完全不同。

C 这个写法看起来更优雅

品味很重要，但它不是事实，换个人就会得出不同的结论。

B 手机用局域网地址打开页面时，浏览器不给加密接口，而我们又要求手机只用浏览器，所以得自己带一份加密代码

它从一条推不翻的事实，加上一个明确的目标出发：局域网地址不是安全上下文，浏览器不给加密接口；手机又只用浏览器。只剩这一条路。



选择题

从“攻击几乎不要钱，处理请求要花内存”这条事实，能推出什么？

- A 先把正文全部读进内存，再检查会话是否存在
- B 先做便宜的检查，比如只看请求头、数连接数，能拒绝就早拒绝
- C 换一台内存更大的电脑



选择题

从“攻击几乎不要钱，处理请求要花内存”这条事实，能推出什么？

A 先把正文全部读进内存，再检查会话是否存在

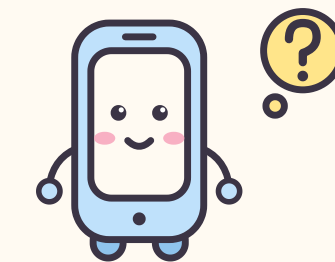
正好反了。不知道密钥的人，也能让你把大正文读进内存。

C 换一台内存更大的电脑

内存再大也是有限的，攻击的成本却几乎为零，这笔账永远算不过来。

B 先做便宜的检查，比如只看请求头、数连接数，能拒绝就早拒绝

越便宜的检查越靠前，每个假请求在你这里只花最少的成本，能走到昂贵步骤的请求越来越少。



选择题

两个一模一样的重放请求同时到达，怎样才能只让一个通过？

- A 检查计数器，放开锁，然后直接登记，中间不再查一次
- B 最后那一次检查和登记，在同一次加锁里完成
- C 在检查之后加一个短暂的等待



选择题

两个一模一样的重放请求同时到达，怎样才能只让一个通过？

A 检查计数器，放开锁，然后直接登记，中间不再查一次

中间有缝。两个请求可能都在登记之前查到“没用过”，都通过。

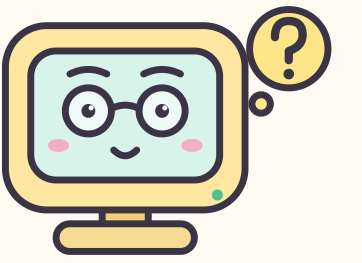
C 在检查之后加一个短暂的等待

等待只会让时间窗口变化，缝还在，只是更难复现。

B 最后那一次检查和登记，在同一次加锁里完成

前面可以有只看不记的预检，但最后一定在同一次加锁里再查一次、马上登记。
第二个请求拿到锁的时候，计数器已经登记过了。

想一想



挑一个你写过或者正在写的项目，写下它推不翻的三条事实，再看看现在的设计，能不能从这三条事实推出来。



想一想

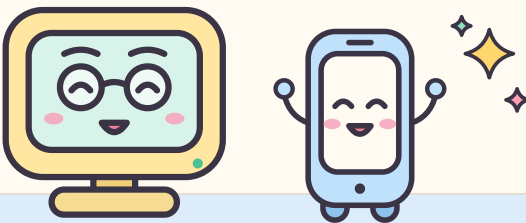
挑一个你写过或者正在写的项目，写下它推不翻的三条事实，再看看现在的设计，能不能从这三条事实推出来。

没有标准答案。常见的发现是：有些设计追不到任何一条事实，是当初照搬的，可以重新考虑；有些事实没有被任何设计照顾到，比如忘了给某个会无限增长的列表设上限，或者某个外部请求没有超时，那就是最该先改的地方。



带走这六条事实，和七个步骤

- ✔ 网络是公共的：加密，画边界，看门的地方要集中
- ✔ 资源有限、攻击不要钱：便宜的检查在前，会增长的有上限，会等待的有时限
- ✔ 很多事同时发生：最后的查和改在同一次加锁里，锁里只算
- ✔ 外部世界不确定、规则确定：核心纯、外壳薄，一件事只有一个真相
- ✔ 人会犯错：独立核对，证明测试会响，改动小、默认安全
- ✔ 最后做判断的是人：照实写边界，把关键决定交给



下一集
全课完结，感谢收看

